Table of Contents

W4111 — Exam Study Guide

MongoDB (`classicmodels`) + Neo4j (sample Movie DB)

How to use this guide

Your professor's exam guidance was explicit:

> *"I strongly advise you to understand the `classicmodels` sample data from homework 3 for MongoDB. You are also responsible for having taken the Neo4j tutorial for the Movie Database that was part of homework 3 setup. You are responsible for being able to query the `classicmodels` data and the Neo4j sample movie dataset. I do provide a summary of the data in the exam, but you should ensure you understand it from the homework. WRT to MongoDB, you are responsible for understanding how to use the following aggregation stages/operators: count, group, limit, lookup, match, project, sort, unwind."*

This guide mirrors that guidance exactly:

1. A **data-model refresher** for both datasets — structure, relationships, and the denormalized shape HW3 loaded into MongoDB.
2. A **stage-by-stage MongoDB cheat sheet** covering only the eight operators the professor listed.
3. A **Cypher / Neo4j cheat sheet** matched to the Movie graph you actually worked with.
4. **Practice problems in HW3 style** — solutions provided — that should feel indistinguishable from M1/M2/M3 and N1/N2/N3.
5. A **common-pitfalls** appendix.

If you can work every practice problem in this guide without peeking, you are in good shape.

Part 1 — The `classicmodels` data (MongoDB)

1.1 The original relational schema

`classicmodels` models a small scale-model retailer. In its native SQL form there are eight tables:

Table	Row count	PK	Important FKs
`offices`	7	`officeCode`	—
`employees`	23	`employeeNumber`	`officeCode`, `reportsTo` *(self-join)*
`customers`	122	`customerNumber`	`salesRepEmployeeNumber`
`productlines`	7	`productLine`	—
`products`	110	`productCode`	`productLine`
`orders`	326	`orderNumber`	`customerNumber`
`orderdetails`	2 996	(`orderNumber`, `orderLineNumber`)	`orderNumber`, `productCode`
`payments`	273	(`customerNumber`, `checkNumber`)	`customerNumber`

Core relationship chain (memorize this — every interesting query traverses part of it):

```
productlines --1:N--> products
                         |
                        N:M (via orderdetails line items)
                         |
orders --1:N--> orderdetails
  |
  N:1
  |
customers --N:1--> employees --N:1--> offices
  |
  1:N
  |
payments
```

1.2 What HW3 actually loaded into MongoDB

HW3 only loads **two collections**, and both are denormalized. This matters because it determines which pipelines even make sense.

`classicmodels.customers` — 122 documents

```
{
  customerNumber: 103,
  customerName: "Atelier graphique",
  contact: { lastName: "Schmitt", firstName: "Carine " },
  phone: "40.32.2555",
  address: {
    addressLine1: "54, rue Royale",
    addressLine2: null,
    city: "Nantes",
    state: null,
    country: "France",
    postalCode: "44000"
  },
  salesRepNumber: 1370,
  creditLimit: 21000.0
}
```

Notes:

- `contact` and `address` are embedded sub-documents. Query nested fields with **dot notation**: `"address.country": "France"`.
- `salesRepNumber` is a foreign key to `employees` — not embedded.

`classicmodels.orders` — 326 documents

```
{
  orderNumber: 10100,
  orderDate: "2003-01-06",          // note: string, not Date
  requiredDate: "2003-01-13",
  shippedDate: "2003-01-10",
  status: "Shipped",
  comments: null,
  customerNumber: 363,              // FK → customers
  orderDetails: [
    { productCode: "S18_1749", quantityOrdered: 30, priceEach: 136.00, orderLineNumber: 3 },
    { productCode: "S18_2248", quantityOrdered: 50, priceEach:  55.09, orderLineNumber: 2 },
    { productCode: "S18_4409", quantityOrdered: 22, priceEach:  75.46, orderLineNumber: 4 },
    { productCode: "S24_3969", quantityOrdered: 49, priceEach:  35.29, orderLineNumber: 1 }
  ]
}
```

Key design facts:

- The relational `orderdetails` table became an **embedded array** inside each `orders` document. That is *the* reason `$unwind` shows up so often.
- There are `2 996` line items total — the number you should see after a single `$unwind "$orderDetails"`.
- Dates are stored as ISO strings like `"2003-01-06"`. Lexicographic sort still yields correct chronological ordering. If you ever need real date math, use `$toDate` first.

Part 2 — MongoDB aggregation cheat sheet

The professor listed exactly these stages/operators: **`$count, $group, $limit, $lookup, $match, $project, $sort, $unwind`**. Learn them cold.

2.1 SQL ↔ MQL mental map

SQL	MQL stage	Note
`WHERE`	`$match`	Put as early as possible (uses indexes).
`SELECT col1, col2 AS x, expr`	`$project`	1 = include, 0 = exclude, or assign an expression.
`ORDER BY`	`$sort`	`{ field: 1 }` asc, `{ field: -1 }` desc. Multi-key preserves order.
`LIMIT`	`$limit`	Always pair with `$sort` for deterministic output.
`SELECT COUNT(*)`	`$count: "n"`	Terminal stage; output is a single doc `{n: ...}`.
(flatten array)	`$unwind`	Turns one doc w/ array of length *n* into *n* docs.
`GROUP BY` + aggregates	`$group`	`_id` is the grouping key; everything else is an accumulator.
`JOIN`	`$lookup`	Always returns an **array** field; usually followed by `$unwind`.

2.2 Each stage, with a `classicmodels` example

`$match` — WHERE

```
{ $match: { "address.country": "France" } }
```

Supports full query-operator syntax: `$eq`, `$ne`, `$gt`, `$lt`, `$gte`, `$lte`, `$in`, `$nin`, `$and`, `$or`, `$regex`, `$exists`.

`$project` — SELECT / reshape

```
{ $project: {
    _id: 0,                          // drop _id
    customerNumber: 1,               // keep as-is
    name: "$customerName",           // rename
    country: "$address.country",     // lift nested
    tier: { $cond: [ { $gt: ["$creditLimit", 100000] }, "gold", "silver" ] }
}}
```

Remember: reference a field value with the `$`-prefix (`"$customerName"`), include/exclude with `1` / `0`.

`$sort` — ORDER BY

```
{ $sort: { status: 1, orderDate: -1 } }
```

`$limit` — LIMIT

```
{ $limit: 10 }
```

`$count` — COUNT(*)

```
{ $count: "nOrders" }   // → [ { nOrders: 326 } ]
```

This is the **stage** named `$count`. There is also an accumulator `{ $sum: 1 }` used inside `$group`. Both exist, both are on the syllabus.

`$unwind` — flatten an array

```
{ $unwind: "$orderDetails" }
```

After this stage, each order document exists once per line item, with `orderDetails` now pointing to a single element (not an array). If an array is empty or missing, the document is **dropped** by default — add `{ preserveNullAndEmptyArrays: true }` to emulate a `LEFT JOIN` effect.

`$group` — GROUP BY + aggregates

```
{ $group: {
    _id: "$status",                    // grouping key (use null for "overall")
    nOrders: { $sum: 1 },              // COUNT(*)
    revenue: { $sum: { $multiply: [ "$orderDetails.priceEach",
                                    "$orderDetails.quantityOrdered" ] } }
}}
```

Common accumulators: `$sum`, `$avg`, `$min`, `$max`, `$first`, `$last`, `$push` (list of values), `$addToSet` (distinct list).

Tip: after `$group` the output field that held your key is named `_id`. Use a downstream `$project` to rename it back (`orderNumber: "$_id"`, `_id: 0`).

`$lookup` — JOIN

Equi-join form:

```
{ $lookup: {
    from: "customers",
    localField: "customerNumber",
    foreignField: "customerNumber",
    as: "cust"
}}
```

`cust` is always an **array** of matched docs (even for 1:1 joins). The idiomatic pattern is:

```
{ $lookup: { from: "customers", localField: "customerNumber",
             foreignField: "customerNumber", as: "cust" } },
{ $unwind: "$cust" },                // or preserveNullAndEmptyArrays: true for LEFT JOIN
{ $project: { _id: 0, orderNumber: 1, customerName: "$cust.customerName" } }
```

2.3 A model pipeline that exercises every required stage

"Top 5 customers by total revenue, with their country and number of orders."

```
db.orders.aggregate([
  { $unwind: "$orderDetails" },
  { $group: {
      _id: "$customerNumber",
      revenue: { $sum: { $multiply: [ "$orderDetails.priceEach",
                                       "$orderDetails.quantityOrdered" ] } },
      nOrders: { $addToSet: "$orderNumber" }
  }},
  { $project: {
      _id: 0,
      customerNumber: "$_id",
      revenue: { $round: ["$revenue", 2] },
      nOrders: { $size: "$nOrders" }
  }},
  { $lookup: { from: "customers", localField: "customerNumber",
               foreignField: "customerNumber", as: "cust" } },
  { $unwind: "$cust" },
  { $match: { "cust.address.country": { $exists: true } } },
  { $project: { customerNumber: 1, revenue: 1, nOrders: 1,
                customerName: "$cust.customerName",
                country: "$cust.address.country" } },
  { $sort: { revenue: -1 } },
  { $limit: 5 }
])
```

That single pipeline uses 7 of the 8 required stages: `$unwind`, `$group`, `$project`, `$lookup`, `$match`, `$sort`, `$limit`. To also exercise `$count`, swap the question to *"how many customers with `creditLimit > 50000` have placed any order?"* — the pipeline is the same shape but terminates with `$count`:

```
db.orders.aggregate([
  { $group: { _id: "$customerNumber" } },
  { $lookup: { from: "customers", localField: "_id",
               foreignField: "customerNumber", as: "cust" } },
  { $unwind: "$cust" },
  { $match: { "cust.creditLimit": { $gt: 50000 } } },
  { $count: "nBigSpenders" }     // terminal stage → { nBigSpenders: <number> }
])
```

Part 3 — The Neo4j Movie dataset

3.1 Schema in one picture (ASCII)

```
                    (:Person)
                  /    |    \    \    \    \
                 /     |     \    \    \    \
         ACTED_IN  DIRECTED  PRODUCED  WROTE  REVIEWED   FOLLOWS
                \    |    /    |     /          \          |
                 \   |   /     |    /            \         v
                 (:Movie)  (:Movie) (:Movie)  (:Movie)  (:Person)
```

Two node labels:

- `(:Movie { title, released, tagline })`
- `(:Person { name, born })`

Six relationship types, **all directed**:

Rel	From → To	Properties
`ACTED_IN`	Person → Movie	`roles: [String]`
`DIRECTED`	Person → Movie	—
`PRODUCED`	Person → Movie	—
`WROTE`	Person → Movie	—
`REVIEWED`	Person → Movie	`summary`, `rating`
`FOLLOWS`	Person → Person	—

Critical gotchas:

- `ACTED_IN.roles` is a **list on the relationship**, not on the node. To get roles use `MATCH (p)-[r:ACTED_IN]->(m) RETURN r.roles`.
- Every movie-related rel points **from Person to Movie** (arrow direction matters for `<-` vs `->` in Cypher patterns).
- `FOLLOWS` is person-to-person (the only rel that isn't movie-bound).

3.2 Cypher cheat sheet

Concept	Syntax
Node pattern	`(p:Person)` — variable `p`, label `Person`
Node + property filter	`(p:Person { name: 'Tom Hanks' })`
Directed rel	`-[:ACTED_IN]->`
Rel variable	`-[r:ACTED_IN]->` then `r.roles`
Any-direction / undirected	`-[:FOLLOWS]-` (matches both directions)
Any relationship type	`-[r]->` (any labeled rel) or `-[*]-` (any rel, any length)
Variable-length path	`-[:FOLLOWS*1..3]->` (between 1 and 3 hops)
Predicate	`WHERE m.released >= 2000 AND m.title CONTAINS 'Matrix'`
Return	`RETURN DISTINCT p.name AS name, count(m) AS n`
Sort / page	`ORDER BY n DESC SKIP 10 LIMIT 5`
Shortest path	`shortestPath((a)-[*]-(b))`
List ops	`collect(m.title) AS titles`, `size(titles)`, `x IN list`, `[x IN list WHERE ...]`

Grouping is **implicit**: any non-aggregated variable in `RETURN` acts like `GROUP BY`.

3.3 MongoDB ↔ Cypher one-liner map

MongoDB	Cypher
`$match`	`WHERE` (or inline pattern filter)
`$project`	`RETURN ... AS ...`
`$sort` / `$limit`	`ORDER BY` / `LIMIT`
`$group` + `$sum/$avg`	implicit grouping + `count()/sum()/avg()`
`$unwind`	`UNWIND listProperty AS x`
`$lookup`	walk the relationship: `-[:ACTED_IN]->`
`$count`	`RETURN count(*)`

Part 4 — MongoDB practice problems (HW3 style)

Every question below is in the style of M1/M2/M3 in your HW3. Solve first, then verify against the solution.

> **Setup assumed** in all solutions:
>
> `python from pymongo import MongoClient client = MongoClient(...) db = client["classicmodels"]`

M-Practice 1 — Filter + projection (M1-style)

Prompt. Produce a list of `customers` whose `country` is `"USA"` **and** `creditLimit` is at least `100000`. Your answer should contain only the fields `customerNumber`, `customerName`, `creditLimit`, and `address.country`.

Answer.

```
filter_ = {
    "address.country": "USA",
    "creditLimit": { "$gte": 100000 }
}
projection = {
    "_id": 0,
    "customerNumber": 1,
    "customerName": 1,
    "creditLimit": 1,
    "address.country": 1,
}
result = list(db["customers"].find(filter_, projection))
```

Why it works. `find()` takes a filter (`WHERE`) and a projection (`SELECT`). Dot-notation `address.country` reaches inside the embedded sub-document for both the filter and the projection.

Validation. Expect a small list (roughly a dozen) — all with `country = "USA"` and `creditLimit >= 100000`.

M-Practice 2 — Aggregation with match + sort + limit

Prompt. Return the 5 customers with the **highest** `creditLimit`. Output fields: `customerNumber`, `customerName`, `creditLimit`. Sort descending.

Answer.

```
pipeline = [
    { "$match":  { "creditLimit": { "$gt": 0 } } },
    { "$sort":   { "creditLimit": -1 } },
    { "$limit":  5 },
    { "$project": { "_id": 0,
                    "customerNumber": 1,
                    "customerName": 1,
                    "creditLimit": 1 }},
]
result = list(db["customers"].aggregate(pipeline))
```

Why it works. Exactly the MQL translation of `SELECT ... FROM customers WHERE creditLimit > 0 ORDER BY creditLimit DESC LIMIT 5`.

M-Practice 3 — Unwind + project (M2-style)

Prompt. Produce one document per **line item** with fields `orderNumber`, `orderDate`, `productCode`, `quantityOrdered`, `priceEach`, `lineValue` (= `priceEach * quantityOrdered`). Sort by `orderNumber`, then `orderLineNumber`.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$project": {
        "_id": 0,
        "orderNumber": 1,
        "orderDate": 1,
        "productCode":     "$orderDetails.productCode",
        "quantityOrdered": "$orderDetails.quantityOrdered",
        "priceEach":       "$orderDetails.priceEach",
        "orderLineNumber": "$orderDetails.orderLineNumber",
        "lineValue": {
            "$round": [
                { "$multiply": [ "$orderDetails.priceEach",
                                 "$orderDetails.quantityOrdered" ] },
                2
            ]
        }
    }},
    { "$sort": { "orderNumber": 1, "orderLineNumber": 1 } },
]
result = list(db["orders"].aggregate(pipeline))
assert len(result) == 2996        # sanity check
```

Why it works. `$unwind` flattens the array so each line becomes its own document; `$project` both keeps top-level order fields and pulls the nested line-item fields up; a computed field is built right inside `$project`.

M-Practice 4 — Group + sum (M3-style)

Prompt. For each `status`, compute: - `nOrders` — the number of orders with that status, - `totalRevenue` — the sum over all line items in those orders of `priceEach * quantityOrdered`, rounded to 2 decimals. Sort by `totalRevenue` descending.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$status",
        "orders": { "$addToSet": "$orderNumber" },
        "totalRevenue": { "$sum": {
            "$multiply": [ "$orderDetails.priceEach",
                           "$orderDetails.quantityOrdered" ]
        }}
    }},
    { "$project": {
        "_id": 0,
        "status":       "$_id",
        "nOrders":      { "$size": "$orders" },
        "totalRevenue": { "$round": [ "$totalRevenue", 2 ] }
    }},
    { "$sort": { "totalRevenue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Why it works. Grouping on `$status` gives the per-status bucket. `$addToSet` collects distinct `orderNumber`s so `$size` gives the true order count (a plain `$sum: 1` here would count *line items*, which is wrong). `$sum` over a `$multiply` expression gives per-status revenue.

M-Practice 5 — Group + $count stage (two ways)

Prompt (a). How many distinct customers have placed at least one order? **Prompt (b).** Confirm the number of orders in the `orders` collection.

Answer (a).

```
pipeline = [
    { "$group": { "_id": "$customerNumber" } },
    { "$count": "nCustomersWithOrders" },
]
# → [ { 'nCustomersWithOrders': 98 } ]
```

Answer (b).

```
pipeline = [ { "$count": "nOrders" } ]   # → [ { 'nOrders': 326 } ]
```

Why it works. `$count` is a terminal stage that outputs one document whose single field is named by the argument string. Combined with `$group`, it counts **buckets**. Used on its own, it counts **documents**.

M-Practice 6 — Lookup (equi-join with a collection of documents)

Prompt. For every order, return `orderNumber`, `orderDate`, `status`, and the placing customer's `customerName` and `country`. Sort by `orderDate` descending. Limit to 10.

Answer.

```
pipeline = [
    { "$lookup": {
        "from": "customers",
        "localField": "customerNumber",
        "foreignField": "customerNumber",
        "as": "cust"
    }},
    { "$unwind": "$cust" },
    { "$project": {
        "_id": 0,
        "orderNumber": 1,
        "orderDate": 1,
        "status": 1,
        "customerName": "$cust.customerName",
        "country": "$cust.address.country",
    }},
    { "$sort":  { "orderDate": -1 } },
    { "$limit": 10 },
]
result = list(db["orders"].aggregate(pipeline))
```

Why it works. `$lookup` writes the matched customer documents into the array field `cust`. `$unwind "$cust"` collapses the array (safe here because each order has exactly one matching customer). Subsequent `$project` lifts the needed fields out with dot notation.

Variation — LEFT JOIN. To keep orders with no matching customer, use `{ $unwind: { path: "$cust", preserveNullAndEmptyArrays: true } }`.

M-Practice 7 — Group across collections (lookup + unwind + group)

Prompt. Compute `totalRevenue` by country. Output `{ country, totalRevenue }`, sorted descending.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$customerNumber",
        "rev": { "$sum": { "$multiply": [ "$orderDetails.priceEach",
                                          "$orderDetails.quantityOrdered" ]}}
    }},
    { "$lookup": { "from": "customers",
                   "localField": "_id",
                   "foreignField": "customerNumber",
                   "as": "cust" } },
    { "$unwind": "$cust" },
    { "$group": {
        "_id": "$cust.address.country",
        "totalRevenue": { "$sum": "$rev" }
    }},
    { "$project": { "_id": 0,
                    "country": "$_id",
                    "totalRevenue": { "$round": [ "$totalRevenue", 2 ] } } },
    { "$sort": { "totalRevenue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Why it works. First group reduces to one doc per customer with their revenue. The `$lookup` attaches each customer's country. A second `$group` re-aggregates by country. Two `$group`s in one pipeline is a perfectly normal idiom.

M-Practice 8 — Customers with no orders (anti-join)

Prompt. List `customerNumber` and `customerName` for customers that have **never** placed an order.

Answer.

```
pipeline = [
    { "$lookup": {
        "from": "orders",
        "localField":   "customerNumber",
        "foreignField": "customerNumber",
        "as": "theirOrders"
    }},
    { "$match":   { "theirOrders": { "$size": 0 } } },
    { "$project": { "_id": 0,
                    "customerNumber": 1,
                    "customerName": 1 }},
    { "$sort":    { "customerNumber": 1 } },
]
result = list(db["customers"].aggregate(pipeline))
```

Why it works. `$lookup` populates `theirOrders` with an array of all their orders (possibly empty). `$match` with `$size: 0` keeps only customers whose array is empty — classic anti-join pattern in MQL.

M-Practice 9 — Top-N within groups (pure pipeline, no joins)

Prompt. For each `status`, find the single order with the largest `totalValue` (sum over its line items). Return `status`, `orderNumber`, `totalValue`. Sort by `totalValue` descending.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": { "status": "$status", "orderNumber": "$orderNumber" },
        "totalValue": { "$sum": { "$multiply": [ "$orderDetails.priceEach",
                                                  "$orderDetails.quantityOrdered" ]}}
    }},
    { "$sort":  { "_id.status": 1, "totalValue": -1 } },
    { "$group": {
        "_id":        "$_id.status",
        "orderNumber":{ "$first": "$_id.orderNumber" },
        "totalValue": { "$first": "$totalValue" }
    }},
    { "$project": { "_id": 0,
                    "status": "$_id",
                    "orderNumber": 1,
                    "totalValue": { "$round": [ "$totalValue", 2 ] } } },
    { "$sort": { "totalValue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Why it works. A compound `_id` `{status, orderNumber}` gives per-order totals within status. Sorting then re-grouping on `status` with `$first` picks the max row per group (this is MQL's standard "argmax per group" idiom).

M-Practice 10 — Full kitchen-sink pipeline

Prompt. "Top 3 customers in France by total revenue. Return customer number, customer name, city, total revenue, and the number of distinct products they have ever ordered."

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$customerNumber",
        "rev":      { "$sum": { "$multiply": [ "$orderDetails.priceEach",
                                               "$orderDetails.quantityOrdered" ]}},
        "products": { "$addToSet": "$orderDetails.productCode" }
    }},
    { "$lookup": { "from": "customers",
                   "localField": "_id",
                   "foreignField": "customerNumber",
                   "as": "cust" } },
    { "$unwind": "$cust" },
    { "$match": { "cust.address.country": "France" } },
    { "$project": {
        "_id": 0,
        "customerNumber": "$_id",
        "customerName":   "$cust.customerName",
        "city":           "$cust.address.city",
        "totalRevenue":   { "$round": [ "$rev", 2 ] },
        "nDistinctProducts": { "$size": "$products" },
    }},
    { "$sort":  { "totalRevenue": -1 } },
    { "$limit": 3 },
]
result = list(db["orders"].aggregate(pipeline))
```

Uses 7 of the 8 required stages: `$unwind`, `$group`, `$lookup`, `$match`, `$project`, `$sort`, `$limit`. If this came up in the exam, changing it to "how many French customers have ever ordered a 'Classic Cars' product?" is just swapping a `$match` condition and replacing the `$sort` / `$limit` tail with `{ $count: "n" }` — which then covers all 8 required stages.

Part 5 — Neo4j practice problems (HW3 style)

All queries assume the sample Movie graph loaded by the `:play movies` tutorial.

N-Practice 1 — Single pattern traversal (N1-style)

Prompt. Return every actor's name and the title of each movie they acted in. Sort by actor then movie.

Answer.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor, m.title AS movie
ORDER BY actor, movie;
```

Why it works. This is the acting equivalent of N1. Mirror the N1 pattern, just swap `DIRECTED` → `ACTED_IN`.

N-Practice 2 — Triangle pattern (N2-style)

Prompt. Compute `director_name`, `movie_title`, `writer_name` for every movie that has both a director and a writer. Sort by `director_name`, then `movie_title`.

Answer.

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[:WROTE]-(w:Person)
RETURN d.name AS director_name,
       m.title AS movie_title,
       w.name AS writer_name
ORDER BY director_name, movie_title;
```

Why it works. Two relationships meeting at the same `(m:Movie)` node — exactly the director/actor pattern from N2 but with `WROTE` instead of `ACTED_IN`.

N-Practice 3 — Co-actor pattern (N3-style)

Prompt. Return the distinct names of all people who acted in a movie with **Keanu Reeves**, ordered by movie title.

Answer.

```
MATCH (k:Person {name: 'Keanu Reeves'})-[:ACTED_IN]->(m:Movie)
      <-[:ACTED_IN]-(co:Person)
WHERE co.name <> 'Keanu Reeves'
RETURN DISTINCT m.title AS movie_title, co.name AS coactor
ORDER BY movie_title, coactor;
```

Why it works. Same "friends-of-friends" pattern as N3; the `WHERE co.name <> ...` filters out the starting node, which otherwise trivially co-acts with itself.

N-Practice 4 — Aggregation (count per group)

Prompt. For each actor, return their name and the number of movies they have acted in. Only return actors with at least 4 movies. Order by count descending, then by name.

Answer.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WITH p, count(m) AS nFilms
WHERE nFilms >= 4
RETURN p.name AS actor, nFilms
ORDER BY nFilms DESC, actor;
```

Why it works. Cypher has implicit grouping: the non-aggregated `p` defines the group. `WITH` is Cypher's way to chain subsequent clauses onto the aggregation result — think of it as the subquery break. Compare to MQL's `$group` + `$match`.

N-Practice 5 — Filter by relationship property

Prompt. List each actor who played the role "Neo", and the movie where they played it.

Answer.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE 'Neo' IN r.roles
RETURN p.name AS actor, m.title AS movie;
```

Why it works. Binding the relationship to variable `r` lets you access the `roles` list that lives on the edge. `IN` is Cypher's list-membership check.

N-Practice 6 — Unwind for per-role rows

Prompt. For each `ACTED_IN` relationship, produce one row per role. Output: `actor`, `movie`, `role`.

Answer.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
UNWIND r.roles AS role
RETURN p.name AS actor, m.title AS movie, role
ORDER BY actor, movie, role;
```

Why it works. `UNWIND` is Cypher's `$unwind` — it flattens a list property into one row per element.

N-Practice 7 — Top-N with limit

Prompt. Return the 5 directors who have directed the most movies, with counts, ordered high to low.

Answer.

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie)
RETURN d.name AS director, count(m) AS nMovies
ORDER BY nMovies DESC, director
LIMIT 5;
```

N-Practice 8 — Subquery / intersection

Prompt. List people who both **acted in** and **directed** the same movie.

Answer.

```
MATCH (p:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(p)
RETURN DISTINCT p.name AS name, m.title AS movie
ORDER BY name, movie;
```

Why it works. Using the same variable `p` on both ends of the pattern forces the director and actor nodes to be the same person.

N-Practice 9 — Variable-length path (Bacon number)

Prompt. Find the shortest path (of any relationships) between Tom Hanks and Kevin Bacon. Return the path.

Answer.

```
MATCH p = shortestPath(
  (t:Person {name: 'Tom Hanks'})-[*]-(k:Person {name: 'Kevin Bacon'})
)
RETURN p;
```

If the exam instead asks for the **length**: `RETURN length(p) AS baconNumber`.

N-Practice 10 — Multi-hop traversal with aggregation

Prompt. For every actor who has acted with Tom Hanks, list their name and the number of movies they share with him. Return top 10.

Answer.

```
MATCH (tom:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
      <-[:ACTED_IN]-(co:Person)
WHERE co <> tom
RETURN co.name AS coactor, count(DISTINCT m) AS sharedMovies
ORDER BY sharedMovies DESC, coactor
LIMIT 10;
```

Why it works. Same co-actor pattern as N3 plus grouping via implicit `GROUP BY co` and `count(DISTINCT m)` to avoid double-counting if the same co-actor appeared in the same movie twice.

Part 6 — Common pitfalls & exam tips

1. **MQL aggregation order matters for performance and correctness.** Place `$match` as early as possible. Don't `$unwind` before `$match` if the match can be done on the parent document — you'll create extra work.
2. **After `$lookup`, always decide: `$unwind` (inner-join) or `preserveNullAndEmptyArrays` (left-join)?** Forgetting this is the #1 cause of "why is my result an array?" bugs.
3. **`$group._id` vs `$project`.** After a `$group`, your key is in `_id`. You almost always want a `$project` afterwards to rename it and hide `_id`.
4. **`$addToSet` vs `$push`.** Use `$addToSet` when counting distinct values; `$push` is the SQL analog of `ARRAY_AGG` (keeps dupes).
5. **`$count` the stage ≠ `$count` in a group.** The **stage** is a terminal counter that outputs `{field: n}`. Inside `$group`, you count with `{ $sum: 1 }`.
6. **Strings vs numbers in `classicmodels`.** `orderDate` is a **string**. Sort works because of the ISO format; arithmetic requires `$toDate`.
7. **Cypher direction is mandatory**. `-[:DIRECTED]->` and `<-[:DIRECTED]-` give different results. Undirected is `-[:DIRECTED]-`.
8. **Grouping in Cypher is implicit.** Every non-aggregated variable in `RETURN` is a grouping key. Want to aggregate on `actor` alone? Only mention `actor` (and aggregates) in the `RETURN`.

9. **Relationship properties live on the edge.** To filter by `ACTED_IN.roles`, give the relationship a name: `-[r:ACTED_IN]->` then `WHERE 'Neo' IN r.roles`.
10. **The "I do provide a summary" clause.** On the exam the professor will paste a schema blurb. Don't ignore it — it's the only source of truth for what collections / labels exist in the version you're being tested on.

Quick last-day review checklist

- [ ] I can write in MQL: any HW3-style question using only `$match, $project, $sort, $limit, $unwind, $group, $lookup, $count`.
- [ ] I can compute total order value via `$unwind + $group + $sum + $multiply`.
- [ ] I can do an **anti-join** ("customers without orders") using `$lookup` + `$match: {arr: {$size: 0}}`.
- [ ] I know what the default `$unwind` does to empty arrays and how to change it.
- [ ] I can write `MATCH (a)-[:REL]->(b)<-[:REL]-(c)` triangles and understand co-actor style queries.
- [ ] I can access relationship properties: `-[r:ACTED_IN]-> ... r.roles`.
- [ ] I can count movies per person with implicit grouping.
- [ ] I know how to pair `WITH` with a filter to emulate `HAVING`.

Good luck!